FORGEFLOW / ENTERPRISE ROADSHOW

把 AI 编程变成可治理的工程交付系统

面向企业 CTO 与研发负责人的 Artifact-driven AI Workflow Framework



确定性编排 · 可审计证据 · 同项目增量 · IDE / CLI 双端一致

THE EXECUTION GAP

AI 提升了代码产出，也放大了交付失控

01

上下文随对话消失

需求、取舍和边界散落在聊天记录中，人员、模型或会话一变，工程记忆就开始漂移。

02

决策缺少责任边界

规划、需求、方案、实现和审查混在同一次生成中，谁有权修改什么并不清晰。

03

结果难以验证与复盘

代码可能能跑，但缺少版本化证据、质量判定、失败回退和可重复的状态恢复。

企业真正缺的不是更强模型，而是一套可治理的 AI 交付协议。

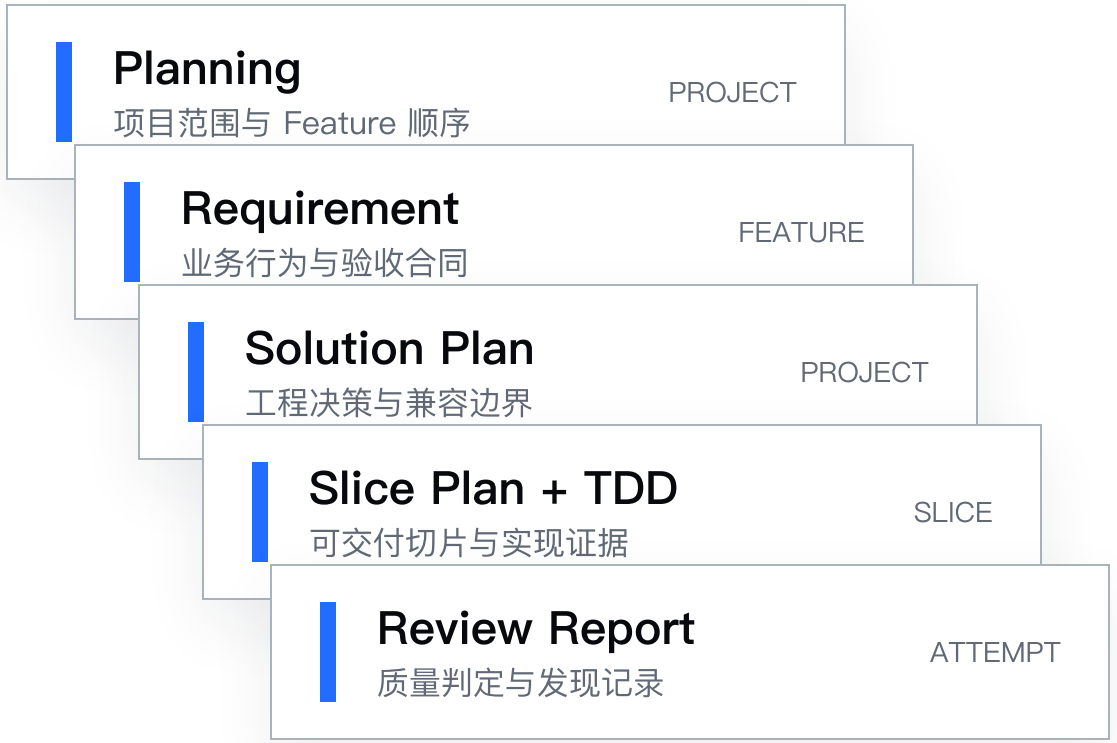
ForgeFlow 把对话变成可验证的 Artifact Graph

用户意图

→ 版本化工程合同

→ 可执行状态

每个工作流只拥有一种核心 Artifact。系统从当前 Artifact Graph 推导下一步，而不是相信对话记忆或模型自述。



THE DELIVERY LOOP

六个工作流组成一条可回退的工程闭环



Plan	Grill	Solution	Slice	TDD	Review
定义项目范围	澄清业务合同	做工程决策	拆成可交付单元	测试驱动实现	质量门禁判定

失败不是“继续生成”，而是携带证据 RETURN 给正确 Owner。

FORWARD / RETURN / TERMINAL

确定性不是口号，而是运行时规则

模型负责推理与创造，
Artifact Graph 负责授权与裁决。

ForgeFlow 将“谁现在可以做什么”从提示词偏好提升为可解析、可拒绝、可恢复的协议状态。

> **图发生矛盾时，系统 HALT；不猜测、不跳步、不静默修补。**

01 状态只来自当前证据

文件路径、版本引用、状态和声明顺序共同定义 active graph。

02 交接必须显式可验证

每次 Handoff 都包含来源、目标、范围、输入 Artifact 与阻塞类别。

03 恢复遵循同一规则

新进程不依赖旧会话记忆，重新读取图并恢复到唯一合法状态。

增量需求只重跑受影响的范围

当前项目

F–01	身份与权限	v1.0
F–02	任务核心流程	v1.0



新增模块
显式进入 Plan

增量之后

F–01	身份与权限	v1.0
F–02	任务核心流程	v1.0
F–03	组织级审计	v1.1

稳定 Feature 保留既有交付证据。 只有 Contract / Engineering Version 变化的范围被重新授权。

QUALITY AS EVIDENCE

质量门禁把 Review 变成可审计证据

POLICY

Markdown 插件注册表

安全、架构、性能、编码规范等规则以可维护的 Markdown 清单注册，无需修改 Python 核心。

DECISION

结构化 Findings

每条发现都绑定严重级别、Owner、Affected Scope、证据和被违反的合同。

TRACEABILITY

不可变历史

Review attempt 与补充报告保留修订历史，替换不会覆盖之前的审查证据。

质量不再是一次“看起来不错”的回答，而是可路由、可复盘、可追责的交付记录。

ONE PROTOCOL, TWO SURFACES

IDE 与 CLI 共用同一套工程协议

IDE：人与 AI 的高带宽协作

```
/forge-plan
/forge-solution
/forge-review
```

适合需求澄清、设计决策和交互式增量规划。

FORGEFLOW PROTOCOL

Artifacts
Handoff
State Resolver
Review Rules

CLI：自动化与可重复执行

```
forge plan
forge run --auto
forge status
```

适合流水线编排、状态查询、恢复执行和显式增量入口。

入口不同，Artifact、版本语义、交接规则和质量门禁保持一致。

THE ENTERPRISE CONTROL PLANE

企业控制面覆盖研发全生命周期

ForgeFlow 不绑定某个模型，也不替代现有研发平台；它补上 AI 参与工程交付时最关键的控制层。

决策

把工程取舍写进合同

需求、方案和范围变化形成版本化 Artifact，而不是留在聊天上下文。

证据

把完成标准绑定到结果

TDD、Review、Findings 与补充报告共同证明一个 Slice 为什么可以通过。

恢复

把失败转化为确定路径

Blocker 与 FAIL Review 路由到明确 Owner，换进程后仍能从图恢复。

A CONTROLLED PILOT

从一个 Feature 开始，四周形成可复制模板

WEEK 01

选择边界清晰的试点

定义业务 Owner、代码范围、质量规则与完成标准，建立第一份 Planning。

WEEK 02–03

跑通完整交付闭环

用真实需求经历 Grill、Solution、Slice、TDD 和 Review，记录阻塞与回退。

WEEK 04

固化团队级工作模板

复盘 Artifact 质量、插件规则和职责边界，形成可推广的项目基线。

试点验收不看“生成了多少代码”

看交付是否可复现、变更是否可解释、失败是否可恢复、证据是否可审计。

WHERE TO START

优先落地在“高协作、高风险、高变化”场景

01

AI 辅助的新产品交付

多人同时使用不同模型时，用 Artifact Graph 统一需求、方案和完成证据。

02

强审计与合规系统

需要解释每次变更、质量判定与责任归属的金融、医疗和企业核心系统。

03

遗留系统持续现代化

按 Feature / Slice 渐进推进，控制跨模块影响，保留不受影响的既有证据。

04

平台型与多团队工程

用统一协议协调业务、架构、开发和质量角色，同时允许项目自定义 Review 插件。

把 AI 编程从个人效率工具，升级为企业交付系统。

ForgeFlow 用版本化 Artifact、确定性 Handoff、范围化增量和可审计 Review，为 AI 研发建立可复制的工程底座。

下一步：选择一个真实 Feature，
启动 ForgeFlow 企业试点。

- 一个明确业务 Owner
- 一个可独立验收的 Feature
- 一组必须通过的质量规则